ISA 444: Business Forecasting

16: PACF and AutoRegressive Moving Average Models

Fadel M. Megahed, PhD

Professor
Farmer School of Business
Miami University

@FadelMegahed
fmegahed
fmegahed@miamioh.edu
Automated Scheduler for Office Hours

Spring 2025

Quick Refresher of Last Class

✅ Define a **stationary time series**.

✅ Understand how plotting techniques can help identify non-stationarity.

✅ Understand how we can **stabilize a non-stationary time series to achieve stationarity** (differencing to stabilize the mean, and log transformation to stabilize the variance).

✅ Understand how to use **unit root tests** to test for stationarity.

Learning Objectives for Today's Class

- Explain the **Partial Autocorrelation Function (PACF)**.
- Explain the **Autoregressive (AR) model**, and how to identify the order of the AR model using the PACF.
- Explain the **Moving Average (MA) model**, and how to identify the order of the MA model using the ACF.
- Explain the **Autoregressive Moving Average (ARMA) model**.
- Understand how to fit AR, MA, and ARMA models using the `ARIMA()` method from the statsforecast Python package.

Revisting the Autocorrelation Function (ACF) and Examining the Partial Autocorrelation Function (PACF)

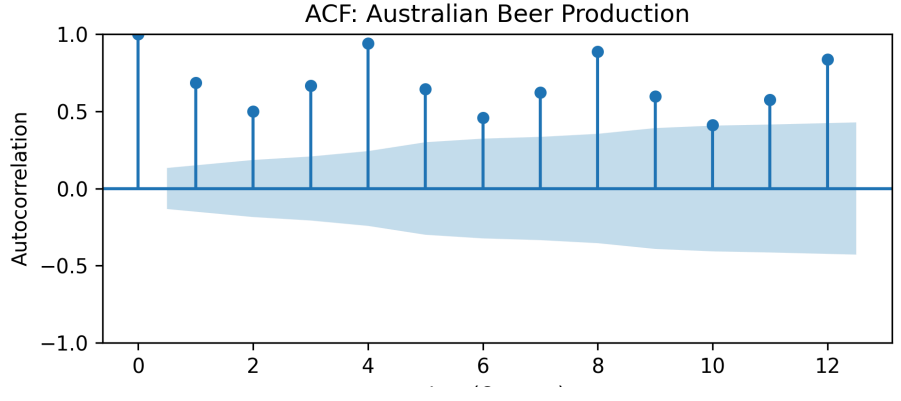
Autocorrelation Function (ACF)

- The autocorrelation is the correlation between a time series and a lagged version of itself.
- The **ACF** is a function that **captures the autocorrelation of a time series at different lags**, which can **help in detecting stationarity and seasonality**.

$$\rho_k = \frac{Cov(Y_t, Y_{t-k})}{Var(Y_t)}$$

```python
import matplotlib.pyplot as plt
import pandas as pd
from statsmodels.tsa.stattools import acf
from statsmodels.graphics.tsaplots import plot_acf

df = pd.read_csv('../../data/aus_beer.csv')

acf_df = pd.DataFrame({
    'lag': range(0, 13),
    'acf': acf(df['beer'], nlags=12)
  })
acf_df.iloc[[1,4], : ] # subsetting lag 1 and 4

plot_acf(df['beer'], lags=12)
plt.title('ACF: Australian Beer Production')
plt.xlabel('Lag (Quarter)')
plt.ylabel('Autocorrelation')
```

```
##    lag      acf
## 1    1  0.68361
## 4    4  0.93997
```



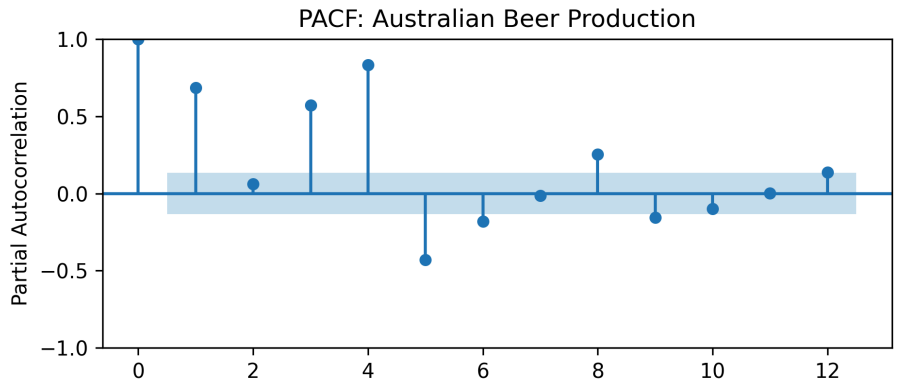
Partial Autocorrelation Function (PACF)

- The **Partial Autocorrelation Function (PACF)** is a function that **captures the autocorrelation of a time series at different lags, while excluding the effect of intermediate lags**.

$$\phi_{k,k} = \frac{Cov(Y_t, Y_{t-k}|Y_{t-1}, Y_{t-2}, \ldots, Y_{t-k+1})}{Var(Y_t)}$$

```python
import matplotlib.pyplot as plt
import pandas as pd
from statsmodels.tsa.stattools import pacf
from statsmodels.graphics.tsaplots import plot_pacf

df = pd.read_csv('../../data/aus_beer.csv')

pacf_df = pd.DataFrame({
    'lag': range(0, 13),
    'pacf': pacf(df['beer'], nlags=12)
  })
pacf_df.iloc[[1,4], : ] # subsetting lag 1 and 4

plot_pacf(df['beer'], lags=12)
plt.title('PACF: Australian Beer Production')
plt.xlabel('Lag (Quarter)')
plt.ylabel('Partial Autocorrelation')
```

```
##    lag      pacf
## 1    1  0.686760
## 4    4  0.877843
```



Comments on the PACF Computation in `statsmodels`

- `statsmodels` uses the **Yule-Walker equations, with sample-size adjustment,** by default.

statsmodels.tsa.stattools.pacf

```
statsmodels.tsa.stattools.pacf(
    x,
    nlags=None,
    method='ywadjusted',
    alpha=None
)                                                                    [source]
```

Partial autocorrelation estimate.

Parameters

x : array_like

Observations of time series for which pacf is calculated.

nlags : int, optional

Comments on the PACF Computation in `statsmodels`

- The use of the Yule-Walker (`yw`) equations in `statsmodels` is a **common approach** to computing the PACF.
- The **sample-size adjustment** in `statsmodels` is used to **correct for the bias** in the sample autocorrelation function estimates.
- The **default** in `statsmodels` is to use the **sample-size adjustment**, hence `yw` is sample-size adjusted.
- Levinson-Durbin (`ld`) showed that the Yule-Walker equations can be rewritten as a set of linear equations that can be solved using a **recursive** algorithm. Hence,
  - `ld` is a **computationally efficient implementation** of `yw`, and
  - `ldb` is **computationally efficient implementation** of `ywm`.
- **On the other hand**, the `pacf()` in R (`stats` package) uses the **biased implementation** the Yule-Walker as its **default**, i.e., R is equivalent to `ywm` or `ldb` in `statsmodels`.

Verifying PACF Computations in `statsmodels` & 

```python
import numpy as np
import pandas as pd
from statsmodels.tsa.stattools import pacf

# generate a random time series
np.random.seed(2025)
ts = np.random.normal(size=100, loc=50, scale=10)

# computing the PACF using different methods
pacf_df = pd.DataFrame({
    'lag': range(13),
    'yw': pacf(ts, nlags=12, method='yw'),
    'ld': pacf(ts, nlags=12, method='ld'),
    'ywm': pacf(ts, nlags=12, method='ywm'),
    'ldb': pacf(ts, nlags=12, method='ldb')
})

# print the results
pacf_df

# save the original ts to reuse in R
pd.DataFrame({'ts': ts}).to_csv(
  '../../data/sim_ts.csv', index=False)

# save the PACF results to reuse in R
pacf_df.to_csv(
  '../../data/pacf_results.csv', index=False)
```

```
##      lag         yw         ld        ywm        ldb
## 0      0  1.000000  1.000000  1.000000  1.000000
## 1      1 -0.117735 -0.117735 -0.116558 -0.116558
## 2      2  0.026043  0.026043  0.025514  0.025514
## 3      3  0.106935  0.106935  0.103698  0.103698
## 4      4 -0.058773 -0.058773 -0.056358 -0.056358
## 5      5  0.026731  0.026731  0.025363  0.025363
## 6      6 -0.056340 -0.056340 -0.052902 -0.052902
## 7      7 -0.043437 -0.043437 -0.040357 -0.040357
## 8      8  0.103929  0.103929  0.095361  0.095361
## 9      9 -0.032238 -0.032238 -0.029049 -0.029049
## 10    10 -0.137204 -0.137204 -0.123049 -0.123049
## 11    11  0.114837  0.114837  0.100998  0.100998
## 12    12  0.052926  0.052926  0.046414  0.046414
```

Verifying PACF Computations in `statsmodels` & R

R Computations and Combining Results

```
ts = read.csv('../../data/sim_ts.csv')
pacf_results = read.csv('../../data/pacf_results.csv')

# r uses biased implementation of Yule-Walker and does not include lag 0
pacf_r = pacf(ts$ts, lag.max=12, plot=FALSE)
pacf_results$r = c(0, pacf_r[[1]]) # prefix w/ lag 0

pacf_results[2:6,]
```

The Combined Results from `statsmodels` and R

```
##   lag          yw          ld         ywm         ldb           r
##     1 -0.11773516 -0.11773516 -0.11655781 -0.11655781 -0.11655781
##     2  0.02604330  0.02604330  0.02551390  0.02551390  0.02551390
##     3  0.10693456  0.10693456  0.10369764  0.10369764  0.10369764
##     4 -0.05877317 -0.05877317 -0.05635773 -0.05635773 -0.05635773
##     5  0.02673134  0.02673134  0.02536253  0.02536253  0.02536253
```

Assumptions of the Yule-Walker Equations

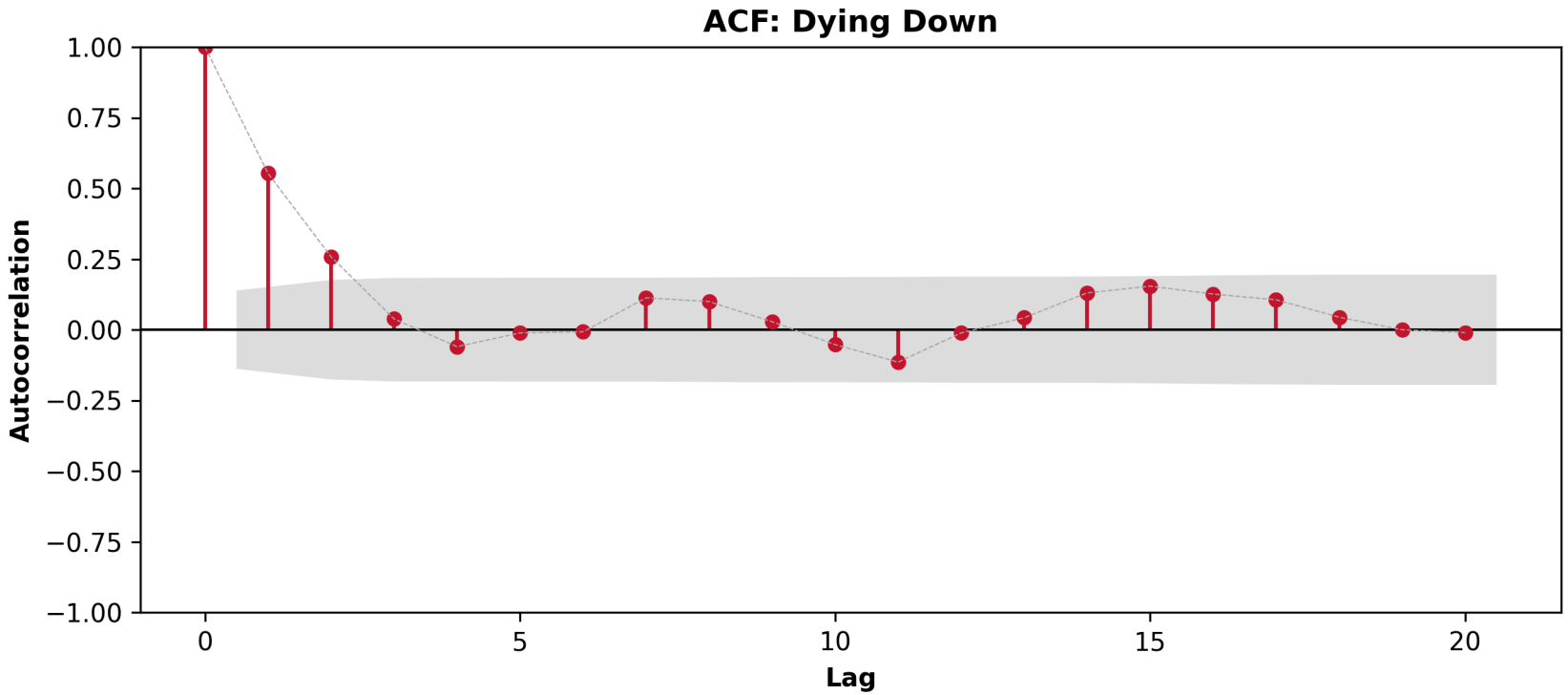
For using the Yule-Walker equations to compute the PACF, the following assumptions are made:

- The time series is **stationary**.
- The time series is in type **AR(p)** form, but the **order p is unknown**.
- Having the prediction p, we use the appropriate **AR(p)** model to estimate the coefficients (i.e., the obtained correlation coefficients are preliminary).
- We will find the estimators for $\phi_1, \phi_2, \ldots, \phi_p$ later using the maximum likelihood method via fitting the AR(p) model.

Source: Łukasz Balbus (2020). Lecture Notes on Econometrics. University of Zielona Góra, Zielona Góra, Poland. Available at: https://staff.uz.zgora.pl/lbalbus/Zdalne%20EDiF%2010.11.2020.pdf.

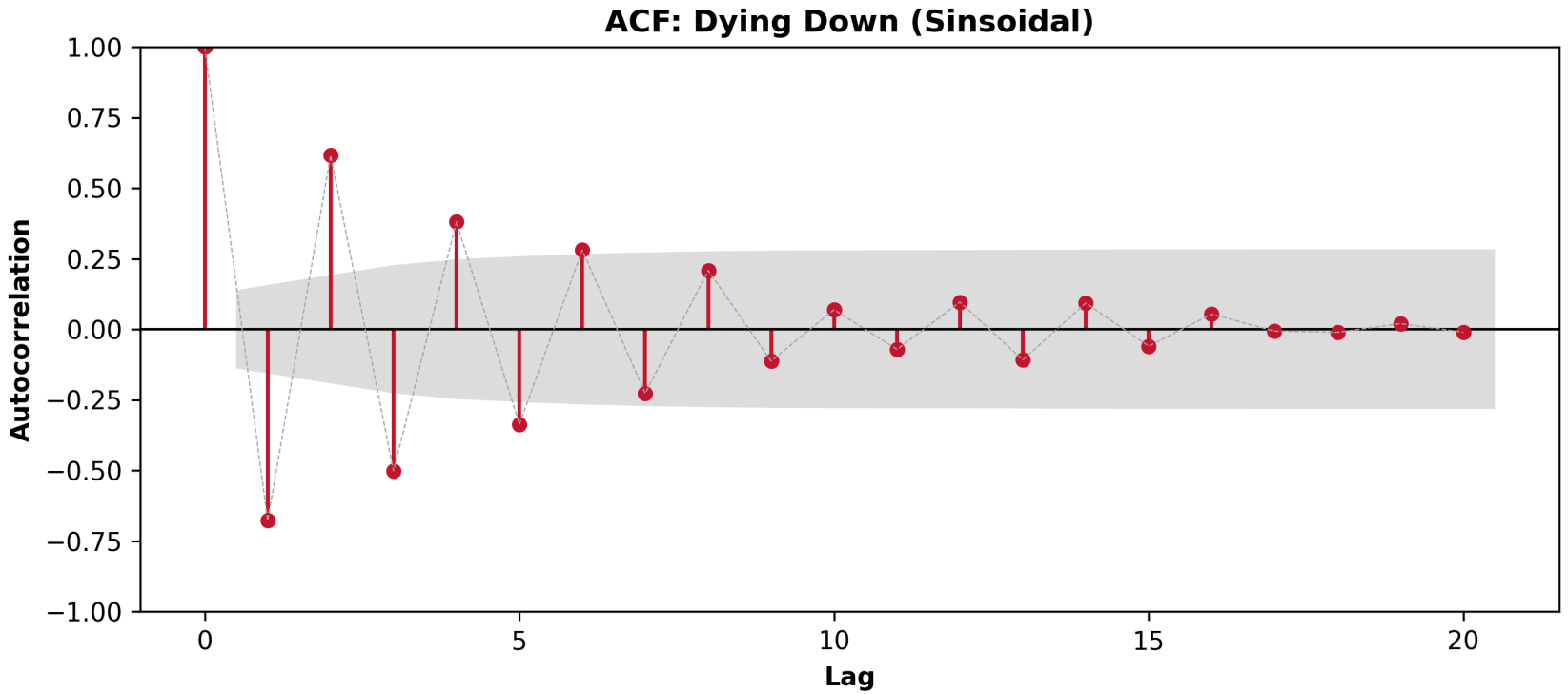
ACF Patterns to Look for: Dying Down

- **Dying Down**: The ACF will **exponentially decay to zero relatively quickly** $\longrightarrow$ this is a sign of a possible **AR** Data Generating Process (DGP).



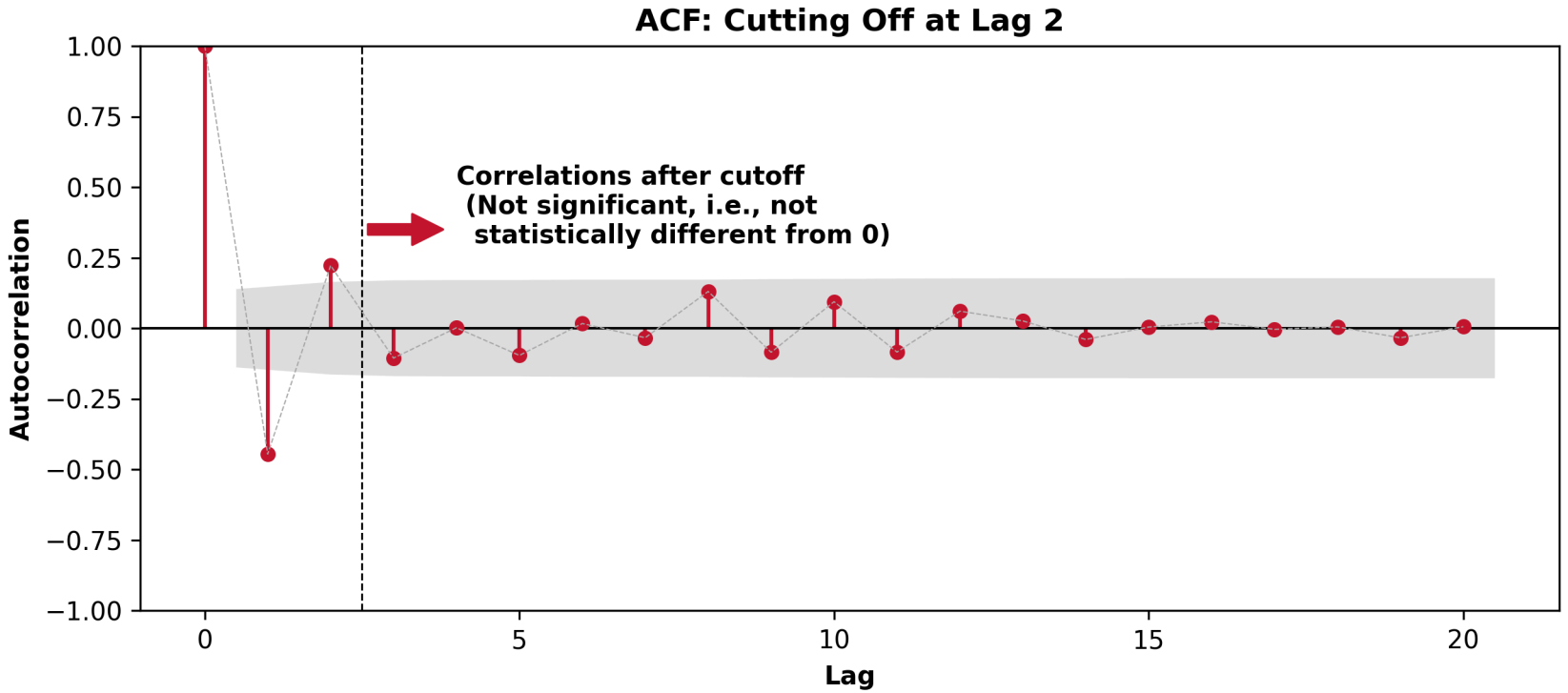
ACF Patterns to Look for: Dying Down (Sinsoidal)

- **Dying Down (Sinsoidal)**: The ACF will **exponentially decay to zero with some oscillations**
  $\longrightarrow$ a possible **AR** DGP, with at least one negative correlation coefficient.



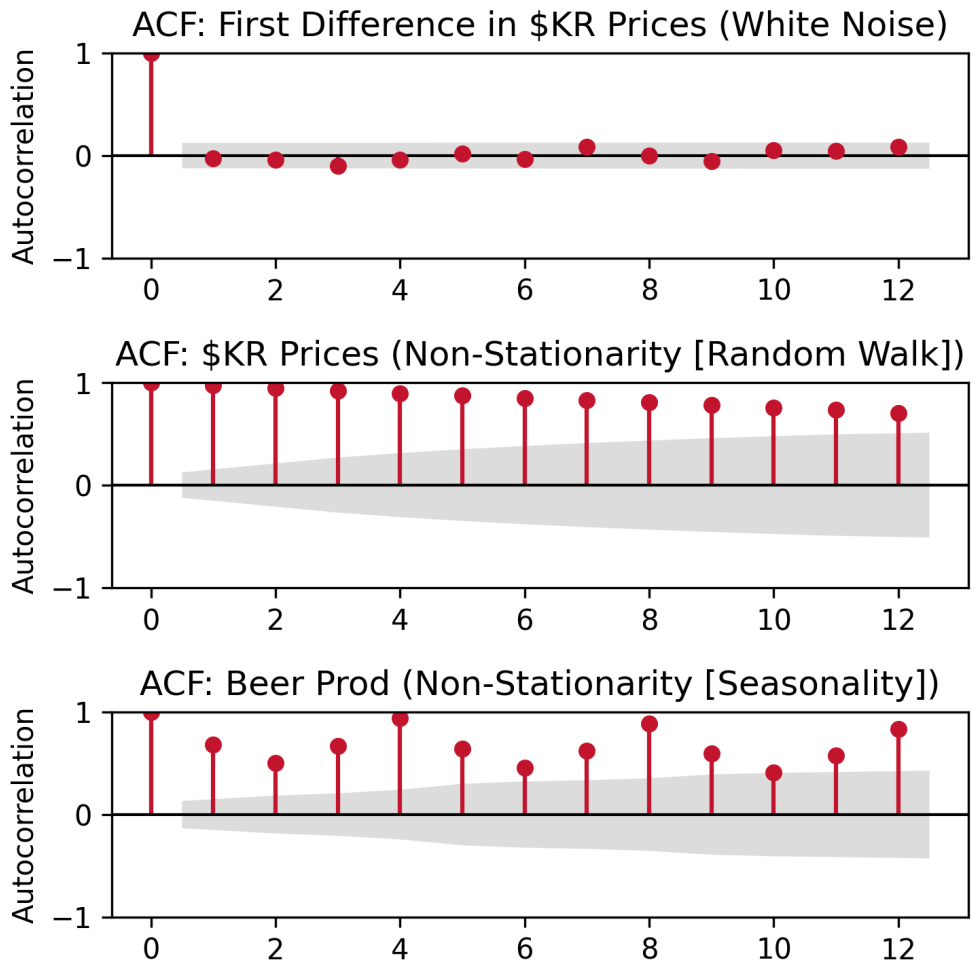
ACF Patterns to Look for: Cutting Off

- **Cutting Off**: The ACF will **cut off after a certain lag** $\longrightarrow$ a possible **MA** DGP.



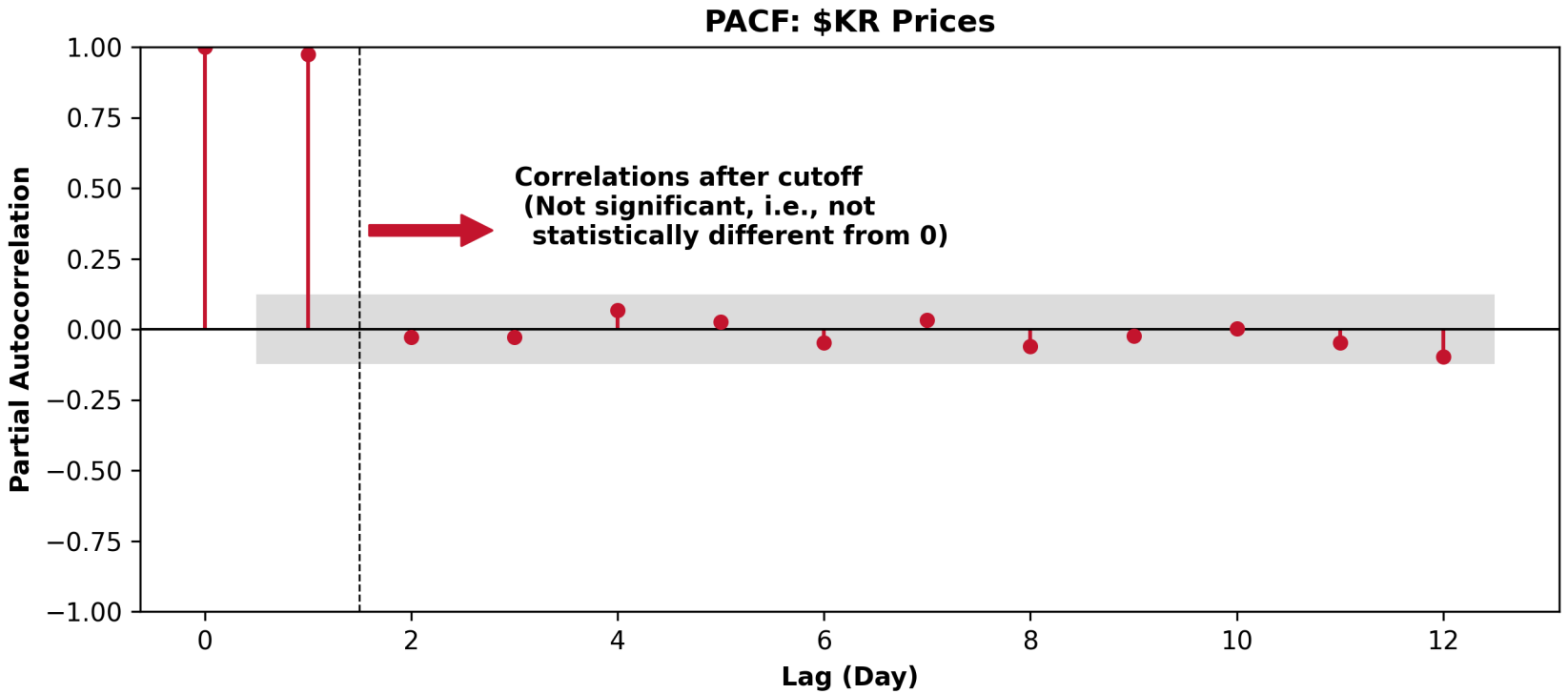
ACF Patterns and Data Generating Processes

- **White Noise:** ACF will **show no pattern.**
- **Random Walk:** ACF will **slowly decay.**
- **Seasonal Patterns:** ACF will **show a pattern at regular intervals.**



PACF Patterns to Look For: Cutting Off

- If the **PACF cuts off after the ACF has decayed down**, this is indicates that an **AR DGP is suitable**. Let us consider the PACF of Kroger's stock price.



Autoregressive (AR) Models

AR Model

In an autoregressive model, we forecast the variable of interest using a linear combination of past values of the variable. The term **autoregressive** indicates that it is a regression of the variable against itself. Thus, an autoregressive model of order p, denoted as AR(p), can be written as:

$$y_t = c + \phi_1 y_{t-1} + \phi_2 y_{t-2} + \cdots + \phi_p y_{t-p} + \epsilon_t,$$

where ϵ_t is white noise.

Note: The AR model **looks similar** to a linear regression model that uses the dependent relationship between an observation and its lagged observations. The **difference** **lies in how their coefficients are estimated in practice.**

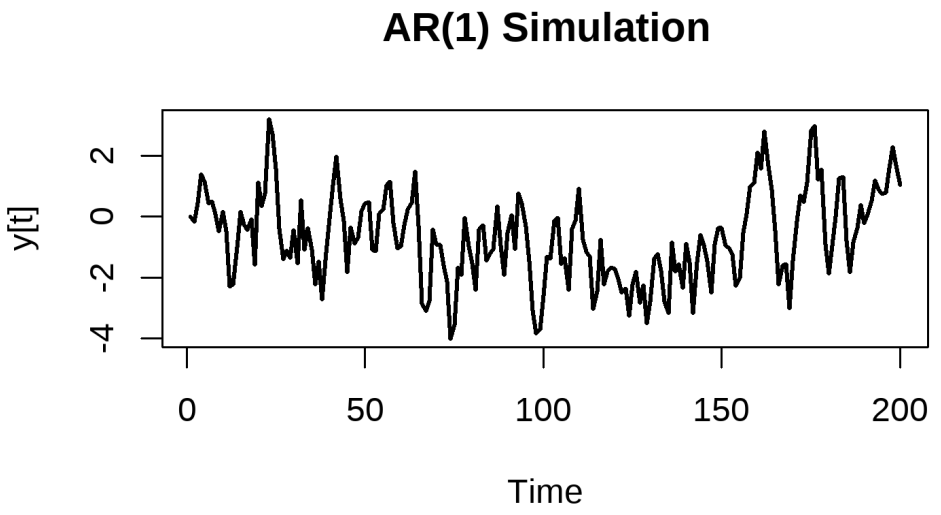
- AR models $\longrightarrow$ **often** utilize MLE while taking into account the time series structure.
- Linear regression $\longrightarrow$ uses ordinary least squares (OLS).

AR (MLE) vs. Linear Regression (OLS)

Since you used R for linear regression, let's illustrate the differences between AR and linear regression using base R.

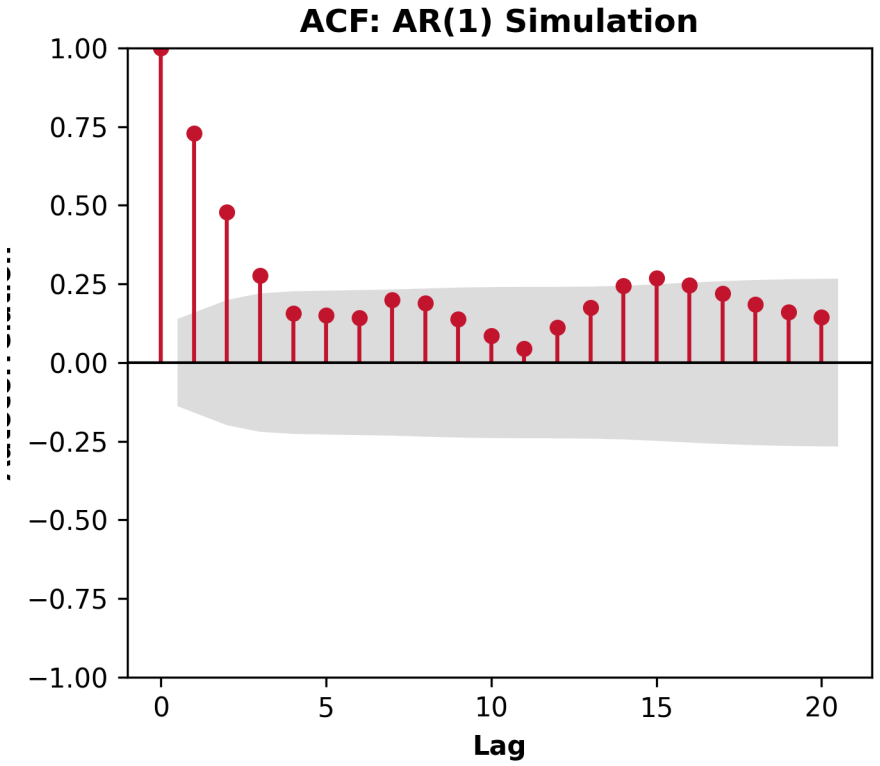
Let us simulate the following AR(1) process: $y_t = -0.2 + 0.7y_{t-1} + \epsilon_t$, where $\epsilon_t \sim N(0, 1)$.

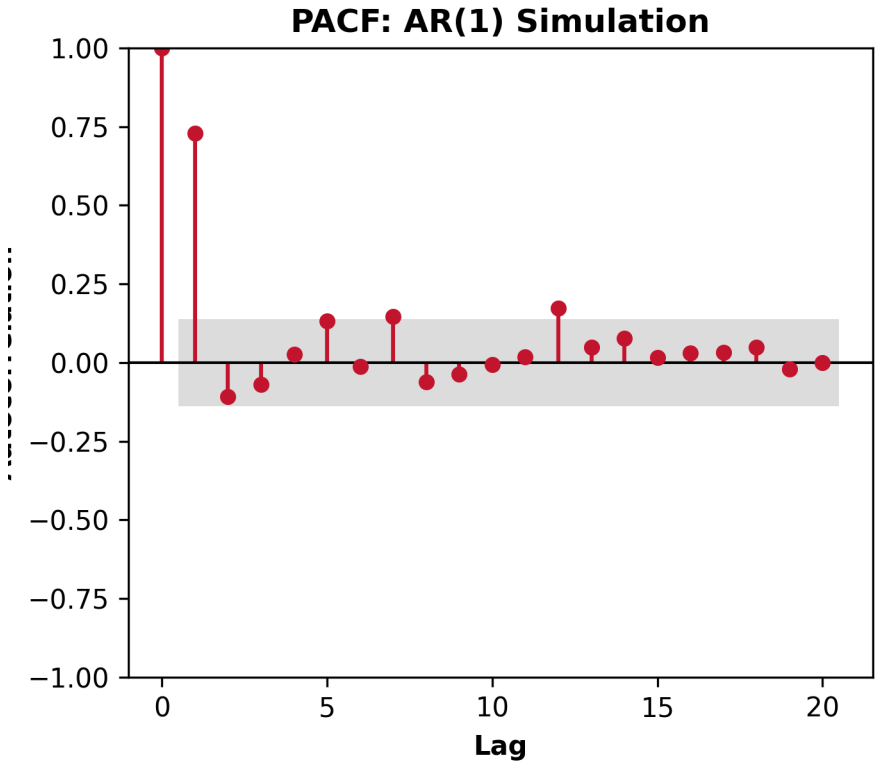
```r
set.seed(2025)

# Simulate y[t] = -0.2 + 0.7 * y[t-1] + error,
n = 200
y = 0
phi = 0.7
intercept = -0.2
errors = rnorm(n, mean = 0, sd = 1)

for (t in 2:n) {
  y[t] = intercept + phi*y[t - 1] + errors[t]
}

plot(
  y, type = "l", lwd = 2, xlab = "Time",
  ylab = "y[t]", main = "AR(1) Simulation"
  )

write.csv(
  data.frame(y), '../../data/ar1_simulation.csv'
  )
```



Simulation: Examining the ACF and PACF

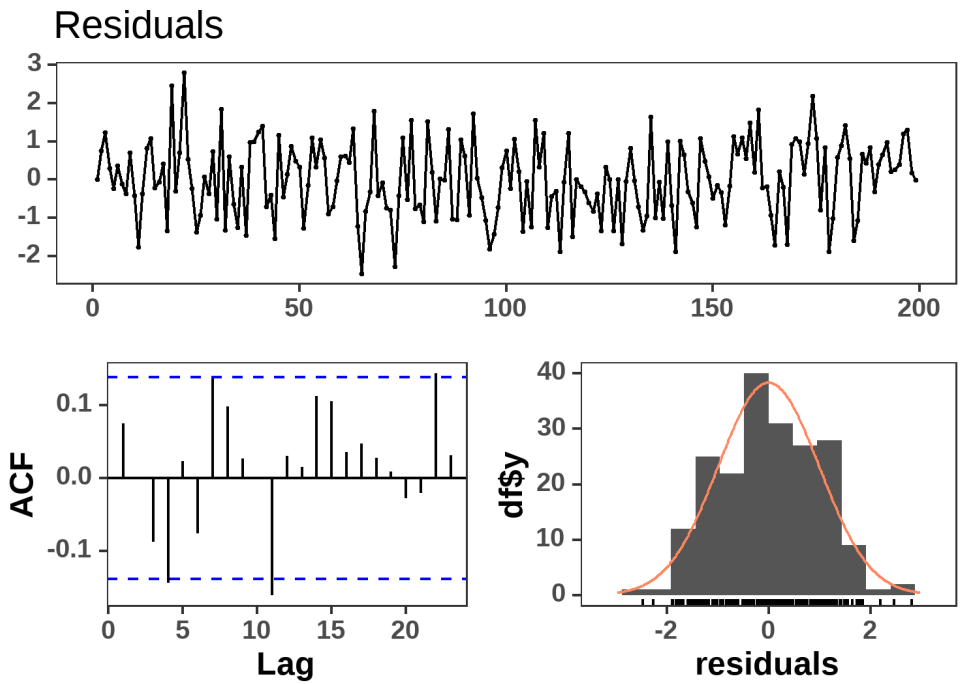
Let us examine the ACF and PACF of our simulated data.





Simulation: The Linear Regression Model (Per ISA291)

```
df = data.frame(y = y)
df$lag1 = dplyr::lag(df$y, 1)
df_reg = na.omit(df)

lm_model = lm(y ~ lag1, data = df_reg)
stargazer::stargazer(lm_model, type = 'html')
```



	Dependent variable:
	y
lag1	0.733***
	(0.049)
Constant	-0.178**
	(0.078)
Observations	199
R^2	0.535
Adjusted R^2	0.532
Residual Std. Error	0.993 (df = 197)
F Statistic	226.325*** (df = 1; 197)
Note:	*p<0.1; **p<0.05; ***p<0.01

Simulation: The AR(1) Model Results

```
from statsforecast.models import ARIMA
from statsforecast import StatsForecast
df = (
  pd.read_csv('../../data/ar1_simulation.csv')
  .assign(
    unique_id = 'simulated_data',
    ds = pd.date_range(start='2024-01-01', periods=200,
  )
)

# typical statforecast workflow
models = [ARIMA(order=(1,0,0))]
sf = StatsForecast(models=models, freq='D')
sf.fit(df)

# getting the model coefficients
# first 0 is for unique_id (only 1 unique_id) and
# second 0 is for the ARIMA model (only 1 model)
phi1 = sf.fitted_[0][0].model_.get('coef')['ar1']
intercept = sf.fitted_[0][0].model_.get('coef')['interc

print(
  'The AR(1) model equation is:\n',
  'y[t] = {:.3f} + {:.3f} * y[t-1] + e[t]'
  .format(intercept, phi1)
  )
```

```
## StatsForecast(models=[ARIMA])
## The AR(1) model equation is:
##  y[t] = -0.758 + 0.730 * y[t-1] + e[t]
```

The AR(1) and the linear regression model are quite similar, but are not identical in this case. The differences arise from the estimation methods used in each model.

Determining the Order of the AR Model

- We can usually tell from the ACF that there is an autoregressive (AR) component to the data because the ACF plot tends to geometrically decrease in magnitude (i.e., "die down").
- The **order** of an AR Process refers to how many lags you include in the autoregressive model.
- Because the ACF of the AR model is a mixture, the **ACF is not useful for determining the order of the AR process.**
- Thus, the **ACF helps us to know that we have an AR model, but not which AR model to fit!**

- The **Partial Autocorrelation Function (PACF)** is used to determine the order of the AR model.
  - For an AR(p) model, the PACF between y_t and y_{t-k} should be 0 $\forall\ k > p$.
  - The PACF will cut off after lag p.

Moving Average (MA) Models

The Moving Average (MA) Model

Rather than using past values of the forecast variable in a regression, a moving average model uses **past forecast errors in a regression-like model**,

$$y_t = c + \epsilon_t + \theta_1 \epsilon_{t-1} + \theta_2 \epsilon_{t-2} + \cdots + \theta_q \epsilon_{t-q},$$

where:

- ϵ_t is white noise; $\epsilon_t \sim \mathcal{N}(0, \sigma^2)$.
- Unlike AR models, **past values of y_t do not appear**; instead, y_t depends on past errors.
- The errors ϵ_t **are unobservable**, meaning they **must be inferred from the data**.

How do we Estimate θ if ϵ_t is Unobserved?

A. Maximum Likelihood Estimation (MLE)

MLE estimates the parameters by maximizing the likelihood function assuming the residuals ϵ_t follow a normal distribution. The process is:

- Assume an initial set of MA parameters $\theta_1, \theta_2, \ldots, \theta_q$.
- Use an **interative algorithm* (e.g., Expectation-Maximization, Kalman Filtering, or numerical optimization)** to estimate the most likely error terms ϵ_t.
- Maximize the likelihood of the observed data given those estimated errors.

B. Nonlinear Least Squares (NLS)

- Instead of directly computing errors, we **minimize the sum of squared residuals using a nonlinear function of parameters.**
- This requires **iterative numerical techniques** (e.g., gradient descent or Newton-Raphson).

Forecasting with MA Model if ϵ_t Is Unobserved?

1. One-Step-Ahead Forecast (t+1):

- Since ϵ_t is unknown in the future, we assume $\mathbb{E}[\epsilon_{t+h}] = 0$.
- The forecast equation uses the most recent residuals $\epsilon_t, \epsilon_{t-1}, \ldots$:

$$\hat{y}_{t+1} = c + \hat{\theta}_1 \hat{\epsilon}_t + \hat{\theta}_2 \hat{\epsilon}_{t-1} + \cdots + \hat{\theta}_q \hat{\epsilon}_{t+1-q}$$

2. Multi-step Forecasts (t+h)

- For **longer horizons**, all future errors are assumed to be zero (i.e., $\mathbb{E}[\epsilon_{t+h}] = 0$ for $h \geq 1$).
- This causes the forecast to **revert to the mean** (c) over time, as past shocks/errors fade out.

Determining the Order of the MA Model

- The **order** of an MA Process refers to how many lags you include in the moving average model.
- The **ACF** of an MA process will show a **sharp cut-off** after lag q (the order of the MA model).
- This cutting off is theoretically related to the **population autocorrelation function** of the MA(q) process at lag k:

$$\rho(k) = \begin{cases} \frac{(\theta_k + \theta_1\theta_{k+1} + \cdots + \theta_{q-k}\theta_q)}{1+\theta_1^2+\cdots+\theta_q^2}, & k = 1, 2, \ldots, q \\ 0, & k > q \end{cases}.$$

- While the equation looks quite complex, the **key takeaway** is that the ACF is not significant (will cut off after lag q) for an MA(q) process.

Autoregressive Moving Average (ARMA) Models

Autoregressive Moving Average (ARMA) Models

Sometimes, if a really high order seems needed for an AR process, it may be better to add one or more MA term. This results in a mixed autoregressive moving average (ARMA) model.

In general, an ARMA(p,q) model is given as

$$\underbrace{y_t}_{\text{Predicted series}} = \underbrace{\delta}_{constant} + \underbrace{\phi_1 y_{t-1} + \phi_2 y_{t-2} + \cdots + \phi_p y_{t-p}}_{\text{Linear combination of Lags of Y up to } p \text{ lags}} + \underbrace{\epsilon_t + \theta_1 \epsilon_{t-1} + \cdots + \theta_q \epsilon_{t-q}}_{\text{Linear combination of Lagged forecast errors up to } q \text{ lags}}$$

The **ACF and PACF of the ARMA(p,q) process exhibit exponential decay exponential decay/ damped sinusoidal patterns**. This makes identifying the order of the ARMA(p,q) difficult.

Model	ACF	PACF
AR(p)	Exponentially decays or damped sinusoidal pattern	Cuts off after lag p
MA(q)	Cuts off after lag q	Exponentially decays or damped sinusoidal pattern
ARMA(p,q)	Exponentially decays or damped sinusoidal pattern	Exponentially decays or damped sinusoidal pattern

Fitting AR, MA, and ARMA Models

- Plot the data over time.
- Do the data seem stationary? If necessary, conduct a test for stationarity.
- Once you can assume stationarity, find the ACF plot.
    - If the ACF plot cuts off, fit an MA(q), where $q =$ the cutoff point.
    - If the ACF plot dies down, find the PACF plot.
    - If the PACF plot cuts off, fit an AR(p) model, where $p =$ the cutoff point.
    - If the PACF plot dies down, fit an ARMA (p,q) model.
        - You must iterate through p and q using a guess and check method starting with ARMA(1,1) models -- increment each by 1.
- Evaluate the model residuals and consider the ACF and PACF of the residuals.
- If model fit is good, forecast future values.

Note: Often you will fit multiple models in Step 3 and compare models in Step 4 to select the best fit.

Fitting AR, MA, and ARMA Models in Python

Let us examine the viscosity data. For this demo, we will only fit ARMA models on the **entire series**, and examine the residuals to see the goodness of fit.

However, it should be noted that **you are also expected to utilize the cross-validation method to evaluate the model's predictive performance (which we do not do in this demo to save time).**

Recap

Summary of Main Points

By now, you should be able to do the following:

- Explain the **Autoregressive (AR) model**, and how to identify the order of the AR model using the PACF.
- Explain the **Moving Average (MA) model**, and how to identify the order of the MA model using the ACF.
- Explain the **Autoregressive Moving Average (ARMA) model**.
- Understand how to fit AR, MA, and ARMA models using the `ARIMA()` method from the statsforecast Python package.

📝 Review and Clarification 📝

- **Class Notes**: Take some time to revisit your class notes for key insights and concepts.
- **Zoom Recording**: The recording of today's class will be made available on Canvas approximately 3-4 hours after the session ends.
- **Questions**: Please don't hesitate to ask for clarification on any topics discussed in class. It's crucial not to let questions accumulate.